โปรเจกต์ประจำวิชา: ภาพรวมและกำหนดการ

โปรเจกต์ประจำวิชานี้คือการนำแนวคิด OOP ทั้งหมดตลอดภาคการศึกษามาใช้ในการออกแบบ สร้าง และสอบป้องกันเกม 2D ด้วย Pygame

โครงสร้างทีมและคะแนน

- **ขนาดทีม**: กลุ่มละ **3 คน** (บังคับ 3 คน; ทำคู่ได้เมื่อได้รับอนุมัติล่วงหน้าเท่านั้น)
- **สัดส่วนคะแนน**: **20%** ของคะแนนรวมทั้งวิชา
- **แพลตฟอร์ม**: Python 3.12+ และ Pygame-ce

กำหนดการสำคัญและการนำเสนอ

- **สัปดาห์ 11 (วันนี้)**: รวมกลุ่ม 3 คนและเลือกแนวคิดเกม
- **สัปดาห์ 15 (18 พ.ย.)**: **นำเสนอผลงาน** และสอบป้องกัน
- **รูปแบบ**: 10–12 นาทีต่อทีม (เดโม/UML 6–7 นาที + สอบปากเปล่า 4–5 นาที)

สรุป

วันนำเสนอผลงาน: 18 พฤศจิกายน 2569 (สัปดาห์ 15) รวมกลุ่ม 3 คนให้พร้อมในสัปดาห์นี้!

เกมของคุณต้องสะท้อนหลักการออกแบบและสถาปัตยกรรมเชิงวัตถุอย่างมีคุณภาพ:

ข้อกำหนด OOP หลัก

- **ขนาด**: ≥ 6 คลาสที่มีความหมายแยกตามโมดูล
- **Encapsulation**: ใช้ Property ตรวจสอบความถูกต้องของสถานะ
- **Polymorphism**: ลูปเกมประมวลผลเอนทิตีหลากชนิดแบบรวมศูนย์
- **ABC**: มีอย่างน้อย 1 คลาสนามธรรมกำหนดสัญญา

ขอบเขตการเล่นเกมและความคิดสร้างสรรค์

- **เปิดกว้างทุกแนว 2D**: แอ็กชัน อาร์เคด พัซเซิล วางแผน
- **ลูปเกมหลัก**: มีเป้าหมายที่ชัดเจน จบเกม และเริ่มใหม่ได้
- **ความคิดสร้างสรรค์**: กลไกเกมที่แปลกใหม่ ธีม หรือสุนทรียภาพ
- **ความเสถียร**: การควบคุมที่ตอบสนองดีและรันได้ราบรื่น

สรุป

เปิดกว้างสำหรับทุกแนวเกม 2D: ผสานสถาปัตยกรรม OOP ที่รัดกุมเข้ากับไอเดียเกมที่สร้างสรรค์!

เกณฑ์การให้คะแนนและการประเมินผล

โปรเจกต์ประจำวิชามีสัดส่วน **20% ของคะแนนรวมทั้งวิชา** โดยประเมินจากเกณฑ์หลัก 5 ด้าน:

การออกแบบ การเล่นเกม และความคิดสร้างสรรค์ (65%)

- **สถาปัตยกรรม OOP (30%)**: ≥ 6 คลาสที่มีหน้าที่ชัดเจน การซ่อนข้อมูล พหุรูป และสัญญา ABC
- **ความสมบูรณ์ของเกม (20%)**: ประสบการณ์การเล่นที่ลื่นไหล ควบคุมตอบสนองดี และไร้บั๊ก
- **ความคิดสร้างสรรค์ (15%)**: แนวคิดเกมแปลกใหม่ กลไกสร้างสรรค์ หรือธีมเฉพาะตัว

การสอบปากเปล่าและชิ้นงาน (35%)

- **การสอบปากเปล่า (25%)**: ความเข้าใจโค้ดรายคน อธิบายสถาปัตยกรรม และตอบคำถามโค้ด AI
- **ชิ้นงานส่ง (10%)**: แผนภาพ UML ไฟล์ `README.md` ไฟล์โครงการครบถ้วน และ `AI_LOG.md`

สรุป

ความคิดสร้างสรรค์มีคะแนนถึง 15%! สร้างเกมที่แปลกใหม่และน่าสนใจที่คุณพร้อมสอบป้องกัน

นโยบายการใช้ AI และมาตรฐานการสอบป้องกัน

เราสนับสนุนให้ใช้เครื่องมือสมัยใหม่อย่างมีประสิทธิภาพ แต่คุณต้องเป็นผู้เข้าใจและควบคุมโค้ดโปรเจกต์ทั้งหมดอย่างแท้จริง:

นโยบายการใช้ AI

- อนุญาตให้ใช้ AI ช่วยระดมความคิด ขึ้นโครงโค้ด และแก้บั๊กได้
- **ต้องมี AI_LOG.md**: ทุกกลุ่มต้องบันทึกคำสั่ง prompt โค้ดที่ได้ การปรับปรุง และการตรวจสอบ
- การใช้ AI โดยไม่เปิดเผยจะถือเป็นการทุจริตทางวิชาการ

มาตรฐานการสอบป้องกันรายบุคคล

- **“คุณต้องเป็นเจ้าของโค้ดที่ส่ง”**: สมาชิกทุกคนจะถูกสุ่มถามโค้ดทุกบรรทัดแยกเป็นรายคน
- ต้องอธิบายหน้าที่ของคลาส การเรียกใช้เมธอด การตัดสินใจเชิงโครงสร้าง และการแก้บั๊กได้ด้วยตนเอง
- ✗ หากตอบไม่ได้ว่าโค้ดที่ AI สร้างขึ้นทำงานอย่างไร จะไม่ได้คะแนนในส่วนนั้น

สรุป

AI อาจช่วยสร้างโค้ดได้ แต่คุณคือผู้ที่ต้องอธิบายและปกป้องสถาปัตยกรรมซอฟต์แวร์

รายการสิ่งที่ต้องส่งและการส่งงาน

ส่งงานโดยอัปโหลดไฟล์ขึ้นระบบ **MS Teams Assignment** ภายในสัปดาห์ที่ 15 (18 พฤศจิกายน 2569):

ไฟล์โค้ดและโปรเจกต์

1. **ซอร์สโค้ด**: โค้ดที่แยกโมดูลชัดเจน รันได้ผ่าน `python main.py`
2. **ไฟล์ Asset**: โฟลเดอร์ `assets/` พร้อมระบุที่มาและสัญญาอนุญาต
3. **เอกสาร**: `README.md` วิธีติดตั้ง ควบคุม สถาปัตยกรรม และ `AI_LOG.md`

แผนภาพ UML และการนำเสนอ

4. **แผนภาพคลาส UML**: รูป `docs/class-diagram.png` แสดงทุกคลาส แอตทริบิวต์ เมธอด ความสัมพันธ์ และ ABC
5. **สไลด์นำเสนอ**: สไลด์สำหรับเดโม 10–12 นาที อธิบายหน้าที่คลาส และตอบคำถาม

สรุป

อัปโหลดไฟล์ซอร์สโค้ด เอกสาร และ UML ขึ้น MS Teams Assignment ก่อนสัปดาห์ที่ 15!

สรุปบทเรียนและคำถามสอบป้องกัน

สรุปประเด็นสำคัญ

- **ABC**: สืบทอดจาก `abc.ABC` เพื่อนิยามสัญญาคลาสที่เป็นทางการ
- **@abstractmethod**: ระบุเมธอดบังคับที่คลาสย่อยต้องเขียนแทนที่
- **Fail-Fast**: โยน `TypeError` ทันทีตอนสร้างออบเจกต์ ไม่ปล่อยให้บั๊กหลุดไปแครช
- **สมดุล**: ใช้ duck typing เพื่อความคล่องตัว ใช้ ABC เพื่อความปลอดภัยของโครงสร้าง

คำถามสอบป้องกันประจำสัปดาห์

คำถาม: ข้อผิดพลาดใดจะเกิดขึ้นเมื่อคลาสย่อยเขียนเมธอดนามธรรมไม่ครบ เกิดขึ้น ณ จังหวะใดพอดี และทำไมสิ่งนี้จึงปลอดภัยกว่า duck typing อย่างเห็นได้ชัด?

เตรียมพร้อมชี้ให้เห็นในโค้ดและอธิบายบรรทัดที่คำสั่งหยุดทำงาน

แหล่งอ้างอิง

Python `abc` Module — เอกสารทางการของ Python เรื่อง Abstract Base Classes และ `@abstractmethod`

PEP 3119 — ข้อเสนอแนะนำ Abstract Base Classes เข้าสู่ภาษา Python โดย Guido van Rossum และ Talin

Python Duck Typing — คำนิยามและปรัชญาทางสถาปัตยกรรมของ Duck Typing ใน Python

UML Class Diagrams — มาตรฐานและสัญลักษณ์แผนภาพคลาสสำหรับคลาสนามธรรมใน UML

เครดิตภาพ

ภาพชื่อเรื่อง: คณะวิศวกรรมศาสตร์ มหาวิทยาลัยธรรมศาสตร์ (แบนเนอร์ TSE)